5、启动优化 ——开发中解决过最难的问题是什么?

Chat iOS 启动流程与优化文档

一、iOS App 启动流程与底层原理

1.1 冷启动（Cold Launch）

冷启动是指 App 不在内存中，需要完整的启动流程，从点击图标到首屏渲染完成的全过程，包括 Pre-main 阶段和 Main 阶段。

1.2 启动阶段详解

Pre-main 阶段（系统层面）

1. 加载可执行文件（Load dyld）

- 内核创建进程，分配虚拟内存空间
- 加载 Mach-O 格式的可执行文件
- 检查文件签名和权限

2. 加载动态链接库（Load dylibs）

- 递归加载所有依赖的 dylib
- 执行 dylib 的初始化代码
- 绑定符号和重定位

3. Rebase & Binding

- Rebase：修正内部指针，调整偏移量（ASLR 地址随机化）
- Binding：修正外部指针，链接外部符号

4. Objc Runtime 初始化

- 注册 Obic 类和分类

- 加载方法列表和协议
- 初始化 Class 的 +load 方法

5. Initializers

- 执行 C++ 静态对象构造函数
- 调用所有标记为 `__attribute__((constructor))` 的函数
- 调用 Objc 类的 +load 方法

Main 阶段（应用层面）

1. main() 函数

```
代码块
main() → UIApplicationMain() → AppDelegate
```

2. AppDelegate 回调

- `application:willFinishLaunchingWithOptions:`
- `application:didFinishLaunchingWithOptions:`
- 初始化三方 SDK
- 创建 Window 和 RootViewController

3. 首屏渲染

- loadView：加载视图
- viewDidLoad：视图配置
- viewWillAppear：视图即将显示
- viewDidAppear：视图已显示（首屏加载完成）

1.3 启动时间计算

Apple 官方定义：

- 冷启动时间 = Pre-main 时间 + Main 阶段时间
- 目标：400ms 以内为优秀，1000ms 以内为合格

测量方法：

- Pre-main：通过 Xcode 的 DYLD_PRINT_STATISTICS 环境变量
- Main：从 `didFinishLaunchingWithOptions` 到首屏 `viewDidAppear`
- 使用 `mach_absolute_time()` 或 `CACurrentMediaTime()` 精确计时

二、通用启动优化手段

2.1 Pre-main 阶段优化

1. 减少动态库数量

- 合并或移除不必要的动态库
- 使用静态库替代动态库（权衡包大小）
- 懒加载非必要的 Framework

2. 优化 Objc 代码

- 减少类和分类的数量
- 避免在 +load 方法中执行耗时操作
- 使用 +initialize 替代 +load（懒加载触发）
- 减少 C++ 静态对象的使用

3. 减少 Rebase & Binding

- 减少 Objc 类、方法、分类数量
- 减少 C++ 虚函数
- 使用 Swift 结构体替代 Objc 类（适当场景）

2.2 Main 阶段优化

1. didFinishLaunching 优化

- 只执行必要的初始化
- 延迟非关键任务到首屏渲染后
- 使用异步线程执行耗时操作
- 避免大量同步网络请求

2. 首屏渲染优化

- 减少 viewDidLoad 中的复杂计算
- 异步加载非关键数据
- 使用占位图快速展示界面
- 延迟加载屏幕外的内容

3. 任务分类与调度

按优先级分类：

- **P0（必须同步）**: 启动崩溃检测、日志系统
- **P1（同步优先）**: 网络配置、数据库、用户状态
- **P2（可异步）**: 三方 SDK、统计上报
- **P3（延迟执行）**: 预加载、缓存清理

2.3 其他优化手段

1. 二进制重排

- 根据启动时调用的方法顺序重新排列
- 减少 Page Fault，提高代码局部性
- 使用 order file 配置

2. 启动图优化

- 使用 LaunchScreen.storyboard
- 避免使用启动图资源文件（减少解码时间）

3. 资源优化

- 压缩图片资源
- 延迟加载大资源文件
- 使用 Asset Catalog 管理资源

4. 网络优化

- DNS 预解析
- 接口合并减少请求次数
- 启动接口降级容错

三、Chat iOS 项目启动流程

3.1 整体架构

代码块

```
AppDelegate
    ├─ TPBootor（启动任务分发器）
    │    └─ TPBootTask（各类启动任务）
```

```
    │        ├─ TPLogBootTask（日志与打点）
    │        ├─ TPGoogleBootTask（Google 服务）
    │        ├─ TPAdBootTask（广告 SDK）
    │        ├─ TPBusinessBootTask（业务配置）
    │        ├─ TPNetworkMonitorTask（网络监控）
    │        ├─ TPRequestBootTask（启动接口）
    │        └─ 其他 BootTask...
    │
    ├─ 基础配置
    │    ├─ 多语言初始化
    │    ├─ 主题系统初始化
    │    ├─ Method Swizzling
    │    └─ MMKV 初始化
    │
    ├─ TPLauncher（启动流程管理）
    │    ├─ 创建 Window
    │    ├─ 设置 RootViewController（登录页/主页）
    │    ├─ syncLaunch（同步任务）
    │    └─ asyncLaunch（异步任务）
    │
    └─ 首屏展示
```

3.2 启动流程详解

第一阶段：启动任务分发（TPBootor）

采用**观察者模式**管理启动任务，将各类初始化逻辑解耦：

设计优势：

- 职责清晰：每个 BootTask 独立封装一类初始化
- 易于扩展：新增启动任务只需实现 TPBootTask 协议
- 统一管理：支持所有生命周期回调（启动、前后台切换等）

主要任务：

- **日志系统**：CmdLog、UTS、神策 SDK 初始化
- **三方 SDK**：Google、Facebook、AppsFlyer、Adjust 等
- **业务配置**：AB 测试注册、后羿开关配置
- **网络监控**：网络状态监听、接口拦截配置
- **启动接口**：用户状态同步、活动弹窗数据

第二阶段：基础配置

在主线程同步执行必要的基础配置：

1. **多语言配置**：根据系统语言或用户设置初始化
2. **主题系统**：初始化主题管理器，支持深色模式
3. **Method Swizzling**：UINavigationBar、UIView 等方法交换
4. **MMKV 初始化**：高性能键值存储
5. **调试工具**：大图检测、图片加载检测（非 Release 环境）

第三阶段：Window 与 RootViewController

1. **创建 Window**：设置根窗口和背景色
2. **判断路由**：
- 用户信息完整 → 进入 TabBar 主页
- 用户信息不完整 → 进入登录/注册流程
3. **显示窗口**：`makeKeyAndVisible()`

第四阶段：同步初始化（syncLaunch）

在主线程执行必须同步完成的初始化：

- **IM 系统启动**：自有 IM 连接与消息注册
- **推送配置**：请求 Device Token
- **用户信息处理**：处理启动参数、恢复用户状态
- **图片缓存配置**：设置缓存策略（3 天过期）
- **AppsFlyer 配置**：归因统计初始化
- **全局样式设置**：导航栏、主题色等
- **内购管理**：验证收据和订单恢复

第五阶段：异步初始化（asyncLaunch）

在后台线程执行非关键任务：

- **用户中心数据拉取**：个人信息、关注列表等
- **敏感词过滤器**：违禁词库初始化
- **性能监控**：卡顿监控、内存监控（可配置开关）
- **设备信息上报**：设备型号、系统版本等
- **磁盘空间检查**：延迟 5 分钟执行（非 Release 环境）
- **全局定时器**：启动周期性任务调度

3.3 关键设计模式

1. 启动任务分发器（TPBootor）

- 使用观察者中心管理所有启动任务
- 统一分发生命周期事件到各个 BootTask

2. 同步/异步任务分离

- syncLaunch：主线程执行，阻塞启动
- asyncLaunch：后台线程执行，不影响首屏

3. 路由懒加载

- TabBar 核心页面预注册
- 其他页面按需注册，支持缓存机制

4. 启动时间监控

- 通过系统 API 获取进程启动时间
- 精确计算从 main() 到启动完成的耗时
- 上报到打点系统进行监控

四、Chat iOS 项目已实施的启动优化

4.1 启动时间监控体系

1. 精确时间测量

- 使用 `AppLaunchTime` 类记录启动耗时
- 通过 `kinfo_proc` 获取进程启动时间戳
- 在 `didFinishLaunchingWithOptions` 开始和结束时打点

- 将启动时长上报到 UTS 打点系统

2. 性能数据收集

- 记录启动阶段的 CPU 占用率
- 记录启动阶段的内存使用情况
- 非 Release 环境显示性能面板

4.2 任务调度优化

1. 启动任务模块化

- 采用 BootTask 分发器模式解耦启动逻辑
- 将日志、SDK、业务配置等拆分为独立任务
- 每个 BootTask 职责单一，便于维护和优化

2. 同步/异步任务明确分离

同步任务（不可延迟）：

- IM 系统初始化（用户状态依赖）
- 推送配置（及时接收通知）
- 图片缓存配置（首屏图片加载依赖）
- 全局样式设置（UI 展示依赖）

异步任务（可延迟）：

- 用户中心详细信息拉取
- 敏感词过滤器初始化
- 性能监控工具启动
- 设备信息上报
- 全局定时器启动

3. 延迟任务策略

- AB 测试注册：登录后才执行
- 磁盘空间检查：延迟 5 分钟执行（非 Release）
- 评分弹窗检查：延迟 1 秒执行
- 首次安装检查：延迟 1 秒执行

4.3 路由系统优化

1. 路由缓存机制

- 通过后羿开关控制路由缓存启用
- 缓存已注册的路由类，避免重复查找

- 减少动态类查找的耗时

2. 核心路由预注册

- TabBar 五个核心页面启动时注册
- 其他页面通过自动注册或按需注册
- 支持自定义路由拦截机制

3. 冷启动路由延迟

- 冷启动时路由跳转延迟 0.2 秒
- 给路由类注册预留准备时间
- 避免启动时跳转失败

4.4 资源预加载与懒加载

1. 图片预加载策略

Feed 列表图片预加载：

- 滚动时预加载可见区域下方 2 个 Feed 的图片
- 只预加载非视频类型的 Feed
- 使用 SDWebImagePrefetcher 批量预加载

图片质量优化：

- 根据展示尺寸请求不同质量的图片
- 多图布局时根据行数选择合适的压缩率

2. 视频预加载策略

短视频预加载：

- 当前播放视频的下一个视频自动预加载
- 预加载队列管理，避免浪费流量
- 缓存达到 10% 后停止预加载

预加载取消：

- 滑动速度过快时取消预加载
- 切换视频时取消之前的预加载任务

3. 图片缓存配置

- 磁盘缓存过期时间：3 天
- 内存警告时自动清理缓存
- 统一的图片加载配置管理

4. 懒加载设计

- UI 组件使用 `lazy var` 延迟创建
- 只在首次访问时初始化
- 减少启动阶段的内存占用

4.5 性能监控与调试

1. 卡顿监控

- 主线程卡顿阈值：2 秒
- 子线程卡顿阈值：2 秒
- 卡顿时采集堆栈并上报到 Firebase

2. 内存监控

- 内存警告时记录 CPU 和内存使用情况
- 上报到打点系统进行分析
- 自动清理图片内存缓存

3. 调试工具（非 Release）

****大图检测：****

- 检测加载的图片尺寸是否超出显示区域
- 帮助发现不合理的图片使用

****图片加载检测：****

- 监控图片加载耗时
- 发现加载性能问题

****性能面板：****

- 实时显示 FPS、CPU、内存
- 开发阶段性能问题快速定位

4. 远程配置开关

- 通过后羿系统控制性能监控开关
- 支持灰度发布和动态控制
- 出现性能问题时可远程开启监控

4.6 代码层面优化

1. Method Swizzling 集中管理

- 启动时统一执行方法交换

- 避免分散在各个模块中
- 减少 Runtime 调用次数

2. 避免启动时的重复操作

- 首次启动标记（isLaunched）
- 区分冷启动和热启动逻辑
- 避免每次 becomeActive 都执行冷启动逻辑

3. 数据库与存储优化

- 使用 MMKV 替代 UserDefaults（性能更优）
- 启动时只加载必要的数据
- 大数据延迟加载或异步加载

4.7 网络优化

1. 启动接口优化

- 通过 TPRequestBootTask 统一管理启动接口
- 只在登录状态下请求
- 失败不影响启动流程

2. 接口合并

- 自动弹窗活动数据在启动接口中返回
- 减少启动阶段的网络请求次数

3. 网络监控

- 启动时就初始化网络状态监听
- 根据网络状态调整资源加载策略

4.8 三方 SDK 优化

1. SDK 懒加载

- 非关键 SDK 在 BootTask 中独立管理
- 可根据需要延迟初始化
- Unity、TikTok SDK 等按需加载

2. SDK 配置异步化

- AppsFlyer 配置在 becomeActive 时才启动
- 等待 ATT 授权后再启动归因

- 避免阻塞启动流程

4.9 其他优化措施

1. 首次启动优化

- 首次安装时记录安装时间
- 标记新老用户，区别对待
- 新用户引导延迟展示

2. 环境差异化处理

- Debug/Inhouse 环境启用调试工具
- Release 环境关闭性能面板和日志输出
- 根据环境加载不同的配置

3. 主题与样式优化

- 启动时只设置全局样式
- 具体页面的主题在页面加载时才应用
- 支持系统深色模式跟随

4. 崩溃防护

- 键盘崩溃修复（通过 Hook 方法）
- 后台任务保护机制
- 启动崩溃检测与上报

五、优化效果与监控

5.1 启动时间目标

Apple 官方建议：

- 优秀：启动时间 < 400ms
- 合格：启动时间 < 1000ms

项目监控指标：

- 通过 UTS 打点系统持续监控
- 按版本、设备型号、系统版本分析

- 建立启动时间基线和告警机制

5.2 持续优化策略

1. 定期 Review 启动流程

- 每个版本迭代检查新增的启动任务
- 评估是否可以延迟或异步化
- 清理废弃的初始化代码

2. A/B 测试验证

- 通过 AB 测试验证优化效果
- 对比不同策略的启动时间
- 选择最优方案推全量

3. 性能回归检测

- 每次发版前检查启动时间
- 对比上一版本是否有退化
- 及时发现和修复性能问题

六、总结

6.1 项目启动优化亮点

1. **启动任务分发器模式**：解耦启动逻辑，职责清晰
2. **精确的启动时间监控**：底层 API 精确测量
3. **同步/异步任务明确分离**：关键任务同步，非关键任务异步
4. **资源预加载与懒加载**：图片、视频智能预加载
5. **路由缓存优化**：减少动态查找耗时
6. **完善的性能监控体系**：卡顿、内存、网络全方位监控
7. **远程配置能力**：通过后羿系统灵活控制功能开关

6.2 后续优化方向

1. **二进制重排**：进一步减少 Page Fault
2. **启动任务优先级管理**：更精细的任务调度
3. **启动流程可视化**：开发工具帮助分析启动瓶颈
4. **更多异步化改造**：将更多同步任务移到异步
5. **启动接口降级容错**：网络异常时不影响启动

6.3 最佳实践建议

1. **新增启动任务时**：优先考虑是否可以延迟或异步
2. **三方 SDK 接入**：评估对启动时间的影响
3. **定期清理**：移除废弃的初始化代码和资源
4. **性能监控**：持续关注启动时间指标
5. **用户体验**：在性能和功能之间找到平衡

文档版本：v1.0

更新时间：2025-11-18

维护团队：Chat iOS 团队